UNIVERSIDADE DA CORUÑA

fic
facultade de
informática
da coruña

Language Modelling
COURSE 2025/2026

Neural models for vector representation

Master in Artificial Intelligence

AUTHORS:

Fernando Baña Amigo ..... fernando.bana@udc.es
Daniel Prieto Díaz ..... daniel.prieto.diaz@udc.es
Alejandro Silva Durán ..... a.silva@udc.es

Contents

1. Introduction

For this practice we are using the 30.000 sentences English news dataset from Leipzig (2024). The objective is to train different embedding layers using a word prediction approach, and then compare their performance of a text classification task using the Reuters dataset from Keras. For the analysis of the embeddings we used the metric of Cosine Similarity and t-SNE visualization against the given target words. To further compare the quality of the trained embeddings, we included a section where we analyze 3 external pretrained embeddings: GloVe, Word2Vec, FastText.

2. Word Embedding Models Training

The Word Embedding models take a context of 2 * Window Size of encoded tokens as input, passes them through an embedding layer, averages the vectors with a GlobalMaxPooling1D, and outputs a softmax distribution of the vocabulary to predict the target word. The target word is the one in the middle of the context, with a Window Size of words previous to it, and after it.

We fixed the vocabulary size to 20.000 words, to create it we used a TextVectorizer from Keras which maps the words to integers. Training used a batch size of 256 and ran up to 20 epochs. A validation split of 10% was used to monitor the performance and avoid overfitting using an early stopping on its loss.

2.1. Hyperparameter Experiments

We tested the following set of hyperparameters:

- Window Sizes: 2, 4, 8
- Embedding Dimensions: 50, 100, 200

Each one of them is trained independently, and the objective is to analyze their differences.

Window Size The window size controls how many surrounding words are used as context. For instance a window size of 2 means we use the 2 previous words and the 2 following words of the target word as context. One important point is that larger windows reduce the number of training samples because there are less valid context-target pairs available. Table 1.1 shows this difference explicitely

Window Size	Number of Training Samples
2	∼470,000
4	∼354,000
8	∼166,000

Table 1.1: Amount of valid context-target pairs generated by window size.

Embedding Dimension controls how many values are used to represent each word. Lower dimensions may not capture enough details for complex relationships, but higher dimensions are more prone to overfitting.

3. Qualitative Analysis

3.1. Cosine Similarity

After training each of the 9 configurations, we computed the top 10 most similar words for each given target word using cosine similarity. This allows us to see what kind of meaning each configuration captures and how hyperparameters change it.

Effect of window size Window size is the most impactful here. Small windows (W=2) make the model focus on immediate neighbours so the embeddings show more syntactic similarity. Larger windows capture more topical similarity, words that appear on the same kind of context. Table 1.2 shows this for the word 'everyone'.

Rank	W=2, D=100	W=4, D=100	W=8, D=100
#1	everybody (0.84)	everything (0.88)	changes (0.93)
#2	me (0.80)	anything (0.87)	work (0.93)
#3	god (0.80)	everybody (0.86)	weve (0.92)
#4	someone (0.79)	anybody (0.86)	difficult (0.92)

Table 1.2: Top 4 similarities across different window sizes of word 'everyone'

At W=2 the model has syntactically interchangeable pronouns (everybody,someone,me). At W=4 the results are indefinite pronouns still semantically correct (everything,anything,everybody). At W=8 the neighbours have no relation at all, instead they reflect words that occur adjacent to 'everyone'.

Overall the best quality appears at W=4, especially for relational words. For instance, "director", W=4 returns chief (0.88), executive (0.85), ceo (0.76). This window is wide enough to capture semantic relations and narrow enough to avoid noise and sparsity that W=8 introduces.

Effect of embedding dimensions Higher dimensions produce more discriminative scores. With this we mean that for words that the models struggle like 'music', the relations are low for all the words. Table **??**, shows this, the more dimensions, the less confident the models are about which words are similar. This could with more dimensions, the models do not have to force unrelated words to share a close space.

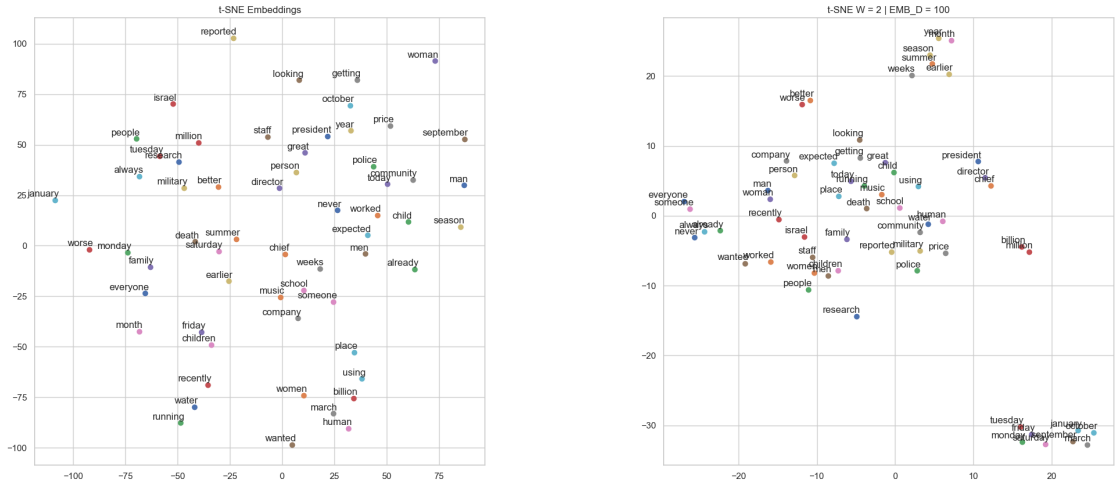
Rank	W=2, D=50	W=2, D=100	W=2, D=200
#1	color (0.77)	color (0.67)	color (0.60)
#2	tunnels (0.76)	diversity (0.66)	kerr (0.59)
#3	enrichment (0.74)	arts (0.66)	furness (0.59)
#4	furness (0.73)	padukone (0.65)	performances (0.59)

Table 1.3: Top 4 similarities for "music" across different dimensions

Overall the dimensions and window size matter less for high frequency words, like 'friday' where all the configurations correctly cluster days of the week together. The dimension has high impact when the words are rare or context dependent.

3.2. t-SNE Visualizations

t-SNE projects the word vectors into 2D, and it is able to preserve local structures. This allows us to confirm what we hypothesized with the cosine similarity analysis. We have made two different visualizations, one of random embeddings before training (Figure 1.1a) and then visualizations for each of the models. In each of them we plot the target words to see their structure in a more visual way than before. To simplify we will only plot the most relevant ones, or the ones we think support better the conclusions we draw



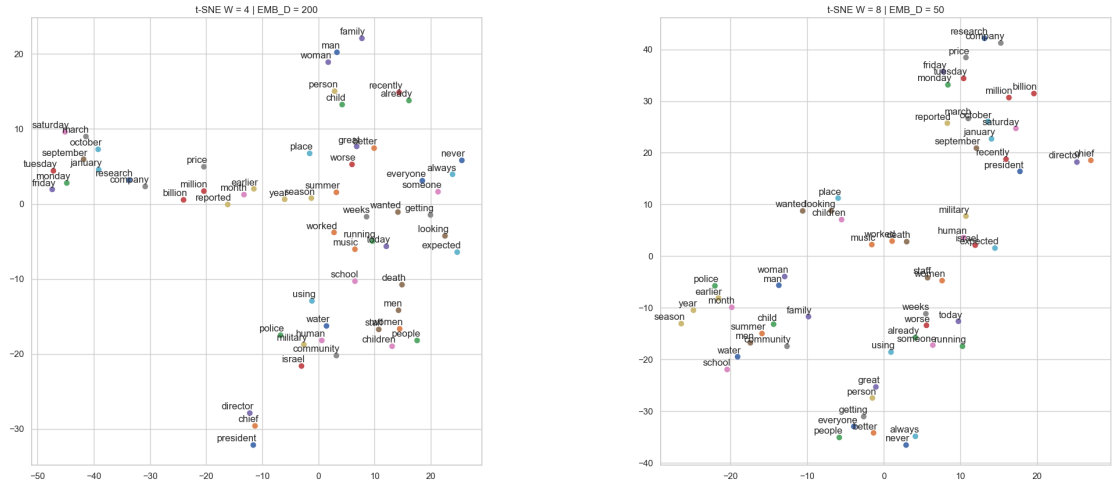
(a) t-SNE Untrained Embeddings

(b) t-SNE for W=2 and D=100

Figure 1.1: Examples of t-SNE visualizations.

Effect of Window Sizes At W=2 (1.1b) we see a few clusters that emerge, the most notable ones are days of the week, clearly isolated on the lower right corner and a temporal cluster (year, month, summer) at the top. However many of the words remain scattered and without clear groupings. At W=4, we see additional clusters: men, women, children more separated then in the W=2, and director, president, chief clustered together as well. The

most refined one is W=4 and D=200 (1.2a), the groups are more distinguishable. At W=8 (1.2b) the groupings degrade a lot, maybe due to lack of training samples or sparsity.



(a) t-SNE for W=4 and D=200

(b) t-SNE for W=8 and D=50

Figure 1.2: Examples of t-SNE visualizations.

Effect of Embedding Dimensions Higher dimensional plots usually show better separated clusters. For instace, using the same W=4, D=50 the central are is crowded with overlapping words, while at W=200 the same clusters are present but cleaner. This reflects that the model is able to encode different types of similarity across more dimensions without hurting the results.

4. Quantitative Analysis - Reuters

The objective here is to see whether the trained embeddings carry useful information for a downstream classification task on the Reuters dataset. We use a simple architecture with: an Embedding layer, a few Convolutional1D layers paired with GlobalMaxPooling, a Dense hidden layer and then a Dense layer with softmax activation of the 46 possible topics. In terms of training configuration we have: SEQUENCE_MAX_LENGTH=256, BATCH=32 and early stopping. We compare three different strategies:

- Baseline: random initiated embeddings, fully trainable from the beginning.
- Frozen pre-trained embeddings: weights from the previously training, loaded and fixed.
- Fine-tunned: same initialization but the embeddings are trainable.

Since the Reuters vocabulary differs from the news corpus, we adapted the embeddings. For each Reuters word, we look up its index in the original vocab and copies the corresponding vector. With 20.000 words, we managed to get 47% coverage, only half of it.

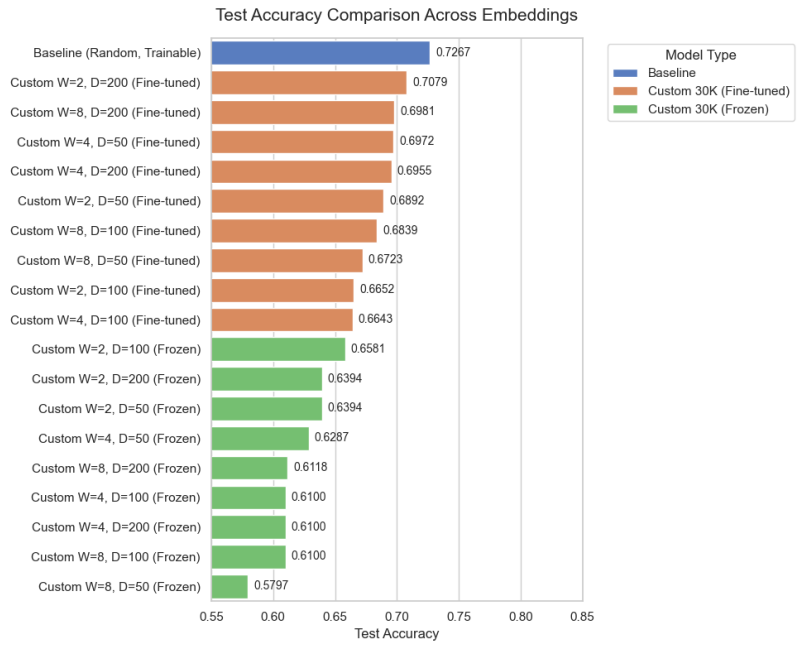


Figure 1.3: Comparison of accuracy on the test set.

Figure 1.3 shows a comparison of the accuracy for all the models. As the plot displays, the **baseline**, which is the reference, is the most performant. It makes sense, because it learns its own representation and has no bias towards the previous task.

Frozen pre-trained models all perform worse than the baseline and the fine tuned models. With accuracies ranging from 0.57 to 0.65. The best result is from W=2 and D=100. In this category we can see a clear pattern, the best models are the ones with the lowest window size. Lower window sizes might transfer better than bigger ones, which are more susceptible to noise. Larger dimensions do not consistently outperform lower ones, which might be due to only 47% of the vocabulary matching, limiting the benefits of richer representations.

Fine-tunning really helps the models close to gap with the baseline. Across all configurations they outperform their frozen counterparts by almost 10 points. The best result now changes and it is W=2 and D=200, so same window size but now a greater dimension allows the model to learn a better representation. The window size dominance previously is not seen here, instead larger embedding dimensions tend to dominate, 3 out of the 4 best models are of D=200. Despite the improvements, no fine-tuned custom model reaches the baseline's performance, which might be due to the big vocabulary mismatch and problem domain shift.

5. Additional Experiments

To further expand the analysis of our trained embeddings, we performed to additional set of experiments. Training the embeddings on more data and comparing against large-scale external embeddings, both on the same classification task.

5.1. Embedding Analysis

The original models were trained on 30K sentences. We will retrain a model of W=4 and D=150 on the 100K sentence corpus to test whether more data helps the models get closer to the baseline.

Figure 1.4 shows the results of the t-SNE analysis after training on the 100K dataset the embedding layer. We can see that they are much cleaner than any of the previous. The socila cluster (man, woman, etc) is well separated as well as the organization (director, ceo, etc) and the calendar words (monday, friday, etc.). This shows that training on more data helps stabilizing the patterns and improve the quality.

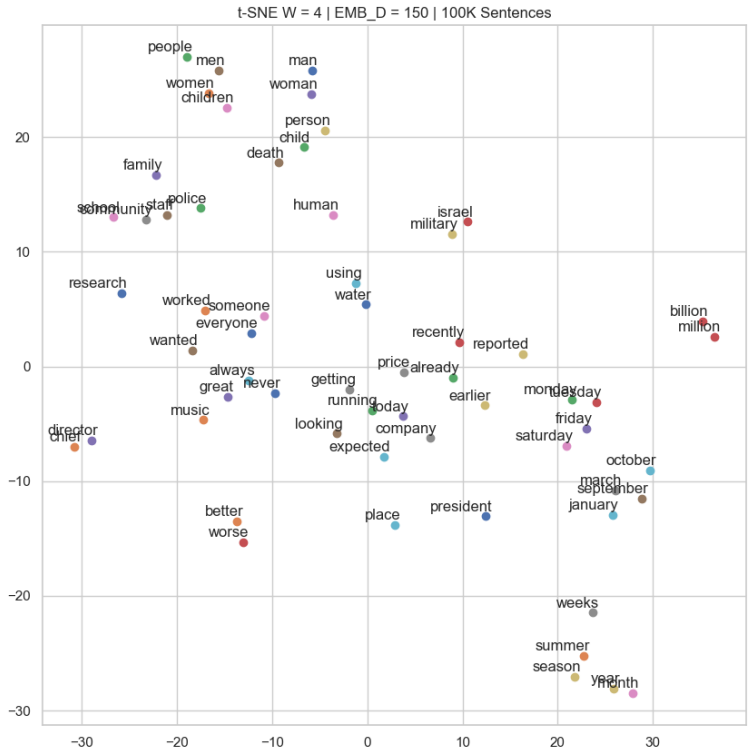


Figure 1.4: Enter Caption

As for the pretrained embeddings, we have analyzed GloVe, Word2Vec and FastText. Figure 1.5 shows the resulting t-SNE plots for them with the target words. GloVe produces the best plot so far, showing clear clusters and good distribution with nearly zero overlap between semantic groups. Word2Vec has similar clusters, but it is less organized than GloVe, most of the words are tighly clustered on the lower region and there is some overlap. FastText is intersting, the clusters are stretched vertically which might be due to the subword training they have.

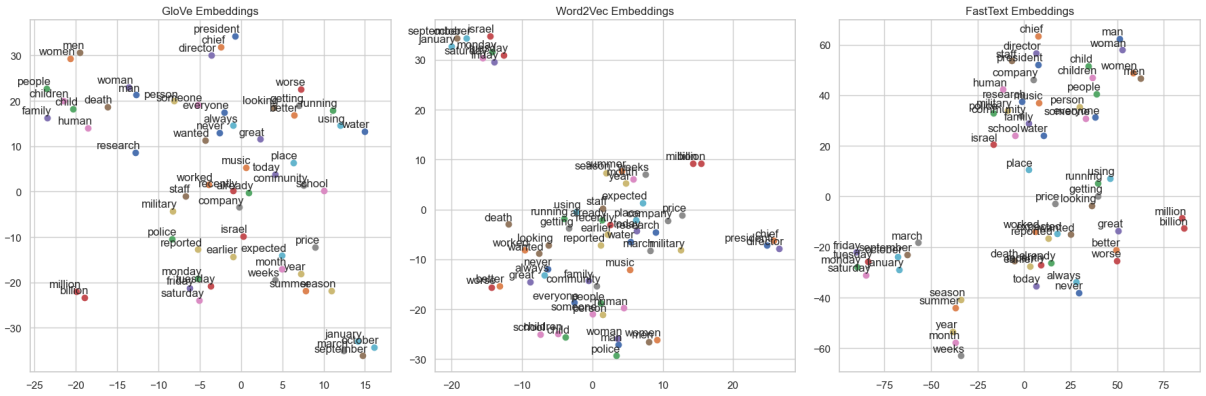


Figure 1.5: t-SNE for the external pretrained embeddings

5.2. Classification Analysis

For the classification task, we used the same architecture and training configurations as previously to make a fair comparison. We trained 3 models with the external pretrained embeddings and then one with frozen and trainable embeddings for the 100K dataset.

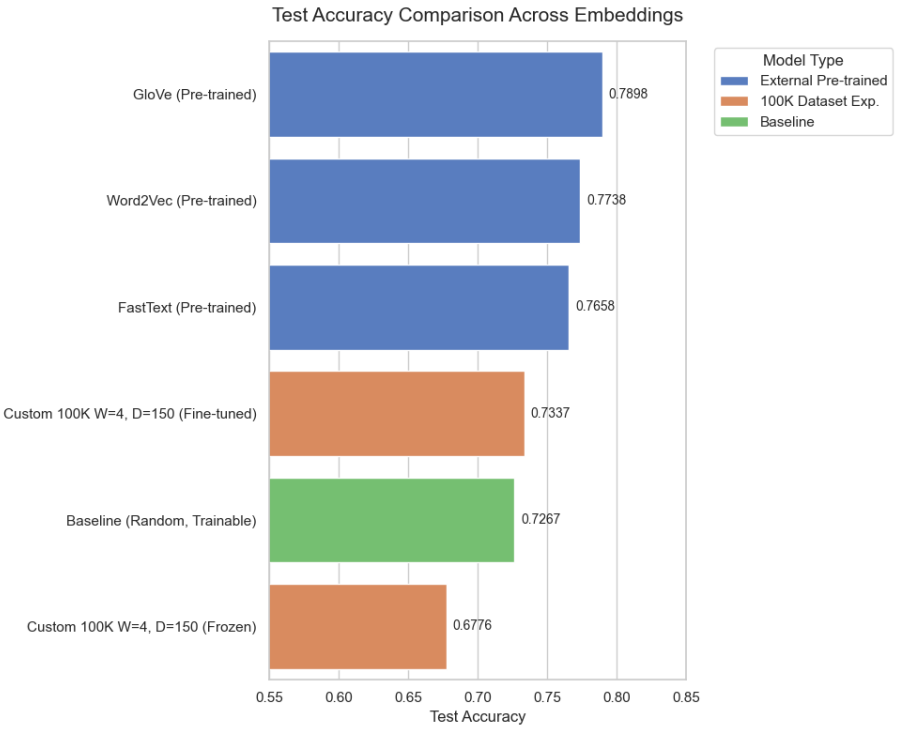


Figure 1.6: Test accuracy of the additional experiments on the Reuters classification

The results are shows on Fig 1.6. We can see that the 100K model reaches a 0.67 accuracy in frozen and 0.73 when fine tunned. The fine-tuned is the first model that exceeds the baseline, it shows that with enough data a custom Word Embedding Model can outperform a from scratch classifier. The frozen counterpart is a bit worse than the baseline, but still better than all the previous frozen models.

The best performing models are the ones with externally pretrained embeddings. This is not surprising, those embeddings are trained or large scale corpus, billions of tokens. So it tells us that the scale of the pretraining corpus play a huge role and it is the dominant factor. Another possible factor is that when comparing the hit rate on Reuters vocabulary, it is much higher than with the embeddings we created. The three of achieve 75%.

6. User Manual

6.1. Submission Files

The source code for this task is organized in a Jupyter Notebook. This format enables the execution of a sequential pipeline (step by step) where all exercises required to complete the assignments are placed. The project contains a set of utils files that are needed to run the notebook. So we have:

- `data/`: folder with the datasets used (30K and 100K)
- `util.py`: set of reusable functions for the project, so the notebook remains more organized
- `LM_Practice.ipynb`: the jupyter notebook with the all the code and experiments in detail
- Some extra folders and files:
  - `embeddings/`: to save all the embedding matrices
  - `models/`: where the classification models are saved
  - `plots/`: for the t-SNE plots
  - `similar_words_tables.txt`: file with all the cosine similarity results for the trained embeddings

The instructions for running the code, mainly the notebook are in the section below. While the code is running, the mentioned folder would be automatically created if not present in order to save the embeddings obtained and the models trained.

6.2. How to run

1. Create a virtual environment with preferred tool or use conda for it
2. Install all the dependencies from the provided `requirements.txt`:

```
pip install -r requirements.txt
```

3a. If using the jupyter environment run:

```
jupyter notebook
```

3b. If using VSCode directly then open there the file.
Open and run `LM_Practice.ipynb`, it contains all the neccessary code.